Pętla while

Pętla while w Pythonie jest jedną z podstawowych struktur sterujących, która pozwala na powtarzanie bloków kodu, dopóki określony warunek logiczny jest prawdziwy. W przeciwieństwie do pętli for, która iteruje po sekwencji, pętla while działa na podstawie warunku logicznego i wykonuje kod tak długo, jak ten warunek jest spełniony.

Działanie:

- Warunek (wyrażenie logiczne) jest sprawdzany przed każdym wykonaniem pętli.
- Jeśli warunek jest prawdziwy (True), blok kodu wewnątrz pętli zostaje wykonany.
- Po wykonaniu bloku kodu, warunek jest ponownie sprawdzany.
- Jeśli warunek staje się fałszywy (False), pętla przestaje działać i program przechodzi do następnej części kodu po pętli.

Składnia:

while warunek_logiczny:

    instrukcje1

else:

    instrukcje2

Jeśli warunek w pętli while nigdy nie staje się fałszywy, pętla będzie działać w nieskończoność. Może to być celowe (np. serwery lub aplikacje w czasie rzeczywistym) lub przypadkowe, co zwykle jest błędem programistycznym

Przykład nieskończonej pętli:

```
while True:
    print("To jest nieskończona pętla")
```

Zakończenie takiej pętli wymaga przerwania działania programu (np. Ctrl + C w terminalu).

Użycie break i continue:

W pętli while można używać instrukcji break i continue do kontroli przepływu:

Break: natychmiast przerywa działanie pętli, wychodząc z niej.

```
licznik = 1
while licznik <= 10:
    if licznik == 5:
```

```
    break  # Przerwij pętlę, gdy licznik osiągnie 5
  print(licznik)
  licznik += 1
```

continue: przeskakuje bieżącą iterację i przechodzi do następnej, bez wykonywania pozostałego kodu w danej iteracji.

```
licznik = 0
while licznik < 5:
  licznik += 1
  if licznik == 3:
    continue  # Pomiń liczbę 3
  print(licznik)
```

pass: instrukcja nie robi nic; można jej użyć, gdy składnia wymaga instrukcji a program nie wymaga działania. Na przykład:

```
while True:
  pass
```

Zad. 1

Napisz program, który będzie kontrolować zużycie paliwa w samolocie. Przed rozpoczęciem każdej jednostki czasu wydrukuj do konsoli informację o pozostałych jednostkach paliwa. Gdy paliwo zostanie wyczerpane, wydrukuj do konsoli informacje 'Konie lotu.'.

Masz do dyspozycji następujące dane.:

- ilość paliwa w samolocie w litrach
- ilość paliwa zużywanego na sekundę w litrach
- czas lotu w sekunadach

Wartości początkowe:

- paliwo = 100
- paliwo_zużyte_na_s = 10
- czas = 0

Zad. 2.

Napisz program, który wydrukuje do konsoli 10 pierwszych liczb pierwszych rozdzielonych przecinkiem tak jak pokazano poniżej.

Pamiętaj, że **liczba pierwsza** - to taka liczba naturalna, która ma dokładnie dwa dzielniki naturalne: jedynkę i samą siebie.

W rozwiązaniu użyj pętli while oraz instrukcji break

Oczekiwany wynik:

2,3,5,7,11,13,17,19,23,29

Pętla for

Pętla **for** w Pythonie, jest instrukcją iteracyjną, służy do iteracji po elementach sekwencji (takiej jak lista, krotka, łańcuch znaków, zbiór lub słownik) lub innego iterowalnego obiektu. Pętla for umożliwia wykonywanie bloku kodu dla każdego elementu iterowalnego obiektu.

for zmienna in **obiekt_iterowalny**: instrukcje	przypisanie elementów obiektu do zmiennej powtarzane instrukcje
else: instrukcje	część opcjonalna - instrukcja jest wykonywana raz po zakończeniu działania pętli, jeśli nie wyszliśmy z pętli za pomocą break
for i **in** range(5)**:** print(i)	Wyświetlanie kolejnych liczb całkowitych w zakresie
for znak in 'spam_napis': print(znak, end=' ')	Wyświetlanie kolejnych znaków napisu

Składnia:

for element in iterowalny_obiekt:

blok kodu do wykonania

element: zmienna, która przyjmuje wartość każdego kolejnego elementu z iterowalnego obiektu.

iterowalny_obiekt: Kolekcja lub sekwencja elementów, przez którą pętla for będzie iterować.

Przykład iteracji po liście:

Kolory = ["czarny", "biały", "czerwony", "żółty", "niebieski", "zielony"]

print(Kolory)

```
for barwa in Kolory:
    print(barwa)
```

Iteracja po łańcuchu znaków:

```
for litera in 'Python':
    print(litera)
```

Zad. 3.

Mając podana listę ulic w danej dzielnicy i wiedząc, że na każdej ulicy znajduje się 5 bloków mieszkalnych, a w każdym z nich jest 10 lokali, wypisz listę wszystkich adresów w dzielnicy.

Lista ulic w dzielnicy:

- Jagodowa,
- Lipowa,
- Kwiatowa,
- Kasztanowa,
- Polna.

Zad 4. dodatkowe

Zamówiłeś w restauracji z grupą x przyjaciół, n potraw (liczbę zamówionych dań i liczbę gości, za każdym razem wskazuje użytkownik), następnie dla każdej potrawy podajesz jej cenę.

Korzystając z pętli while napisz program, który pozwoli obliczyć średnią cenę zamówionej potrawy.

Podziel sprawiedliwe rachunek miedzy wszystkich gości.

Rodzaje zakresów

Funkcja **range()** w Pythonie jest używana do generowania sekwencji liczb, które można wykorzystać w pętlach, takich jak pętle for. range() zwraca obiekt sekwencji liczb całkowitych, który jest używany głównie w pętlach for do iterowania przez liczby.

range(n)	zwraca sekwencję liczb całkowitych od 0 do n-1 z krokiem	range(4)
range(n, m)	zwraca sekwencję liczb całkowitych od n do m-1 z krokiem 1	range(5,10)
range(n, m, k)	zwraca sekwencję liczb całkowitych od n do m-1 z krokiem k	range(5,10,2)

Składnia funkcji range()

Funkcja range() może być używana w trzech podstawowych formach:

range(stop)

range(start, stop)

range(start, stop, step)

1. range(stop)

Generuje sekwencję liczb od 0 do stop - 1.

Argumenty:

stop: Końcowa wartość sekwencji (nie włączona).

np:

```
for i in range(5):
    print(i)
```

2. range(start, stop)

Generuje sekwencję liczb od start do stop - 1.

Argumenty:

start: Początkowa wartość sekwencji.

stop: Końcowa wartość sekwencji (nie włączona).

np:

```
for i in range(4,16):
    print(i)
```

3. range(start, stop, step)

Generuje sekwencję liczb od start do stop - 1, przy czym wartości są inkrementowane o step.

Argumenty:

start: Początkowa wartość sekwencji.

stop: Końcowa wartość sekwencji (nie włączona).

step: Rozmiar kroku między wartościami (domyślnie 1).

np:

```
for i in range(1, 10, 2):
    print(i)
```

Przykład zastosowania w praktyce:

```
# Przykład użycia w pętli for
for i in range(1, 101):  # Generuje liczby od 1 do 100
    print(i)

# Przykład z krokiem
for i in range(100, -1, -1):  # Generuje liczby od 100 do 0, co 1
    print(i)
```

Należy zwrócić uwagę, że jeśli start jest większy niż stop i step jest dodatni lub jeśli start jest mniejszy niż stop i step jest ujemny, wynikowy obiekt range będzie pusty,

Ponadto funkcja range() jest pamięciooszczędny, ponieważ generuje liczby tylko wtedy, gdy są potrzebne.

Zad. 5

Napisz kod, który generuje listę liczb zmniejszających się o 4, zaczynając od 80 i kończąc w 0.

Zad. 6

Napisz pętlę nieskończoną, w której użytkownik podaje liczby całkowite. W przypadku liczby ujemnej, następuję wyjście z pętli.

Zad. 7 dodatkowe

A)

Grupa laboratoryjna składa się z n studentów (wartość n podaje użytkownik). Wprowadzamy liczbę punktów dla każdego studenta.

Napisz program, który obliczy średnią liczbę punktów w grupie z wykorzystaniem pętli while.

Zastosuj instrukcje continue w programie tak, aby po podaniu liczby punktów spoza przedziału 0 – 100 pomijać dalsze wykonywanie pętli. Sprawdź działanie programu.

B)

Następnie zmień pętlę na nieskończoną, czyli taką której warunek zawsze jest prawdziwy. Zakończ działanie pętli przy użyciu instrukcji break.

Obie wersje zadnia proszę zamieścić w sprawozdaniu.

Łańcuchy

Dzięki użyciu trzech podwójnych (lub pojedynczych) cudzysłowów z każdej strony ciągu można tworzyć bezpośrednio wielolinijkowe ciągi znaków a także zachowywać wcięcia w tekście:

print("""To jest

wielolinijkowy

ciąg znaków""")

Wewnątrz łańcucha znaków można umieszczać bezpośrednio cudzysłowy, ale innego rodzaju, niż ten, który wyznacza początek i koniec ciągu

print('Autorem książki "Trzej muszkieterowie" był Alexandre Dumas.')

Można też umieścić w ciągu znaków ten sam rodzaj cudzysłowu, jak ten, który otacza łańcuch, należy je jednak w takim przypadku poprzedzić znakiem lewego ukośnika \:

print("\"Monty Python i Święty Graal\" to komedia filmowa z roku 1975.")

Stosując \ możemy wprowadzać do ciągów tekstowych znaki specjalne, takie jak tabulatory (\t) czy znaki nowej linii (\n). Jeśli chcielibyśmy wydrukować np. \t nie jako tabulator ale taki właśnie ciąg znaków, \ należy poprzedzić także znakiem \, wtedy jest on traktowany jako zwykły znak:

print("\tTo jest tekst z tabulatorem,\n"

"to jest nowa linia,\n"

"a tak można \"uciec\" od znaku \"\\\": \\t")

Ciekawostki Print

Spacja jest domyślnym separatorem, ale można to zmienić dodając na końcu listy argumentów: sep="_"

Wyświetlanie liter

Tekst[i] – i-ta litera

Tekst [i : j] – od indeksu i do j

Tekst [i: j: k] - od indeksu i do j z krokiem k

Tekst [-i] – i-ta od końca

Zad. 8

Wyświetl z łańcucha tekst ="Python jest super" znaki o indeksach:

• Zerowym

• Ostatnim

• Co drugi, zaczynając od zerowego

• Co trzeci zaczynając od pierwszego

• Od dziesiątego do ostatniego

• Wyświetl od końca do początku

Zad. 9

Napisz programy, które:

• Wczyta (zmienną) imię oraz wyświetli tekst „Witaj” oraz wczytane imię.

• Wczyta (zmienną) wiek oraz wyświetli tekst „Twój wiek to: ” oraz wczytany wiek.

• Wczyta (zmienne) imię i nazwisko i wyświetli inicjały.

• Wczyta łańcuch i wyświetli go pięć razy.

• Wczyta dwa łańcuchy, a w trzecim łańcuchu zapamięta połączone te dwa łańcuchy.

• Wczyta dwa łańcuchy, a w trzecim łańcuchu zapamięta pierwszą połowę